

Ficha de Exercícios

Exercícios de introdução

1. Escreva uma aplicação que gere um inteiro aleatório entre 0 e 100, inclusive, e que permita ao utilizador adivinhar o número gerado, através de indicação sucessiva de valores.
2. Escreva uma aplicação que peça ao utilizador para pensar num número entre 1 e 100, inclusive. Através de várias perguntas ao utilizador, o qual responderá 'ACERTOU', 'PENSEI NUM NÚMERO MAIOR' ou 'PENSEI NUM NÚMERO MENOR', a aplicação tentará adivinhar o número. No final deve ser apresentado o número de tentativas que foram necessárias para adivinhar.
3. Escreva uma aplicação de consola que crie um *array* de inteiros, perguntando ao utilizador a dimensão do *array* e quais os valores dos seus elementos. Com base no *array* criado desenvolva funções *static* para...
 - a) Imprimir o *array*.
 - b) Calcular e imprimir o maior e o menor elemento presentes no *array*.
 - c) Somar todos os elementos do *array*, retornando a soma.
 - d) Calcular e imprimir a média dos valores do *array*.
 - e) Inverter todos os valores presentes no *array*, $[i_0, i_1, \dots, i_{n-1}] \Rightarrow [i_{n-1}, \dots, i_1, i_0]$
4. Defina uma classe que contenha, como membro, um *array* de inteiros com 20 elementos. Os valores a incluir neste *array* são inteiros aleatórios entre 0 e 100, inclusive. Um inteiro gerado aleatoriamente apenas deve ser colocado no *array* caso esse valor ainda não esteja armazenado, isto é, o *array* não deve conter valores repetidos. Usar uma função membro booleana (que devolve um valor booleano) para verificar se um determinado valor existe no *array*. Para cada valor gerado, deverá ser chamada a função para verificar se o mesmo já existe no *array*. Se ele existir, o valor deve ser descartado e deve ser gerado um novo valor.
A classe deve ter funções para:
 - a) Listar os 20 valores do *array*.
 - b) Mostrar a quantidade de valores que foram gerados em duplicado, ou seja, a diferença entre o total de valores gerados e o número de valores aproveitados (no caso concreto serão 20).
5. Defina uma classe, *Aposta*, que represente uma aposta no Totoloto. Uma aposta é definida por cinco números inteiros compreendidos entre 1 e 49, todos diferentes entre si e não necessariamente introduzidos de forma crescente, e um "número da sorte" compreendido entre 1 e 13 (pode ser igual a um dos 5 números indicados

anteriormente). A classe quando é construída não possui qualquer número “preenchido”, devendo ser disponibilizados métodos para:

- Preencher um número de cada vez com vista à construção da aposta. O método deve possuir dois parâmetros: o primeiro é o número a associar à aposta, garantindo que não há valores repetidos, o segundo será um valor booleano para indicar se o número pertence à aposta principal ou se corresponde ao “número da sorte”.
- Verificar se a aposta está completa (cinco números e “número da sorte”).
- Preencher automaticamente uma aposta completa.
- Comparar a aposta corrente com uma chave ganhadora. A chave sorteada é passada através de dois parâmetros: array de 5 valores inteiros e um valor adicional correspondente ao “número da sorte”.
- Fazer *reset* da aposta, eliminando todos os números eventualmente introduzidos até então.

Crie uma aplicação que utilize esta classe, permitindo a criação de instâncias da classe `Aposta` e a utilização de todas funcionalidades referidas nas alíneas anteriores.

- Escreva uma aplicação que calcule e imprima a transposta de uma matriz. A matriz deverá ser representada através de classe adequada e deve possuir métodos adequados para as funcionalidades referidas.
- Escreva uma aplicação que some matrizes retangulares. Uma matriz deverá ser representada através de uma classe adequada. A soma deverá ser realizada através de duas formas distintas:
 - Função membro que acumulará aos valores de uma matriz os valores de outra matriz.
 - Função estática que recebe duas matrizes e retornará uma nova matriz com o resultado da soma, ou seja, não se pretende que sejam alteradas as matrizes originais.
- Defina uma classe que contenha, como um membro, uma matriz de $m \times n$ elementos. Esta classe deve ter funções para alterar os elementos da matriz, calcular a soma de cada linha, calcular a soma de cada coluna, calcular a soma de todos os seus elementos e imprimir toda a informação.

Exemplo de output para uma matriz definida programaticamente:

Matriz:

1	0	2	-1	3
4	3	2	1	0
1	-2	3	4	5
8	5	1	3	2

Soma das linhas: 5, 10, 11, 19

Soma das colunas: 14, 6, 8, 7, 10

Soma total: 45

9. Escreva uma aplicação que calcule e imprima o triângulo de Pascal. O triângulo de Pascal deverá ser representado através de uma classe própria, que deverá incluir funções para: gerar triângulo com uma determinada profundidade, gerar uma *String* representativa do triângulo e mostrar o triângulo na consola. Quando o objeto for criado deve ser indicado através do *construtor* a profundidade pretendida.

V1:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
.....
```

V2:

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
.....
```